实验五 简易计算器设计

一、实验目的

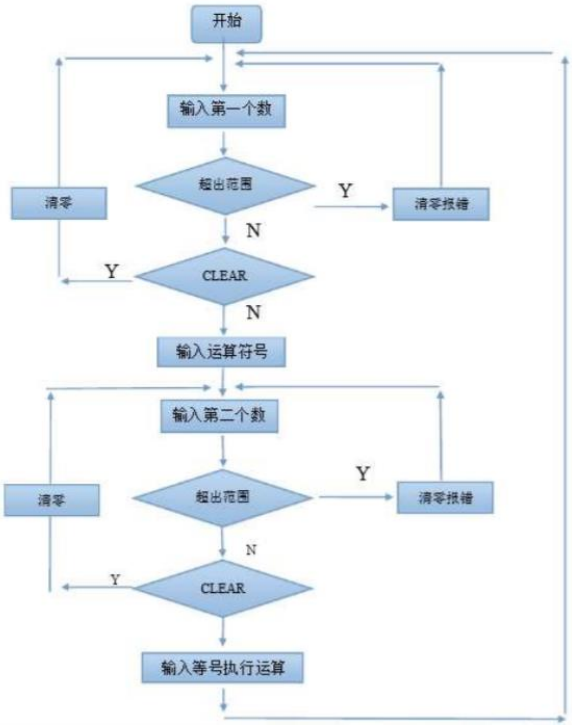
1. 理解任务管理的基本原理，掌握 μCOS-II 中 任务管理的基本方法；
2. 掌握 μCOS-II 中任务管理的基本方法；
3. 掌握嵌入式系统中 LCD 与键盘控制的一般 方法。

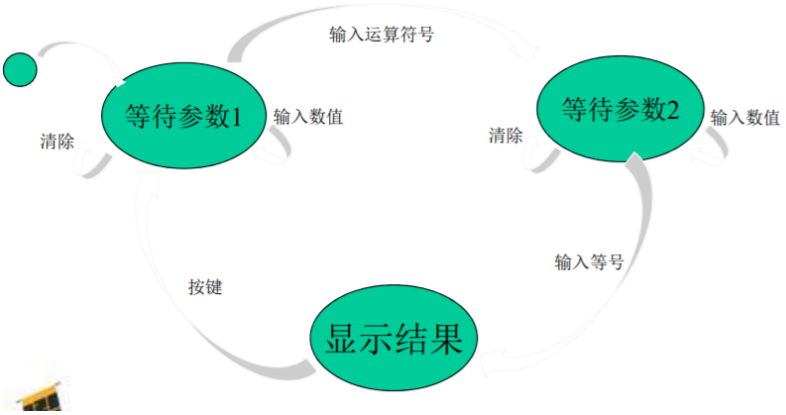
二、实验内容

1. 设计多个应用任务，用其中一个任务控制 LED 灯的状态；
2. 设计多个应用任务，用其中一个任务读取 键盘键值，键盘的响应用中断实现；；
3. 实现一个简易的计算器 Task1：键盘、Task2：流水灯、Task3：LCD 显示

三、实验原理

1. 计算器读取数据和计算流程图





从上图可以推导出计算器的设计步骤。

首先第一步（也就是任务一）利用键盘的读取功能，读入操作数 1，这里可以自行设计功能，比如我设计计算器是可以读取多位十进制数，如果读取到字母‘A’代表着删除上一位输入的数字，并且继续输入，如果读取到字母‘B’，代表重新输入操作数 2，之前的输入都作废，如果读取到字母‘F’代表着，输入结束，进入下一个任务。

第二步（也就是任务二）是利用键盘的读取功能读取操作符，很明显他需要和第一步（也就是任务一）进行同步工作，所以这里需要用到信号量来进行任务同步通信。如果读取到的字符是‘+’或‘-’或‘*’并且读取到‘F’表示结束输入，进行到下一个任务，如果不是这三个字符，那么系统任务字符无效，继续进行读取。如果读到字母‘B’表示重新输入，字符作废。

第三步（也就是任务三）是利用键盘的读取功能读取操作数 2，这里的功能是按照任务一来进行设计的，只是在读取到‘F’字母时比任务一多了一个任务就是要完成操作数 1 和操作数 2 以及操作符的运算并且将结果存储下来。

任务进行中时，根据实验要求，需要设计多任务运行，所以我还设计了一个 led 闪烁的任务（也就是任务 4）。同时根据实验要求，还有一个任务（任务 5）是利用 lcd 屏幕进行展示，这里我将在输入操作数和操作符过程时，由于是多任务运行，所以会不断刷新 lcd 屏幕来展示表达式输入进度，会展示目前操作数的输入过程，比如在输入操作数 2 时输入了 12，那么此时屏幕就会显示操作数 1 操作符 以及 12，会不断刷新显示屏幕，实时显示输入进度。

四、实验步骤

1. 验证示例源码

- 拷贝整个实验例程源码目录到本地磁盘自己的工作目录下；
- 使用 µVision IDE for ARM 通过 ULINK2 仿真器 连接实验板，打开实验例程目录 04-uCOS\3.1_LED_test 子目录下的 ucos2.Uv2 例程，编译链接工程；
- 将程序下载到实验平台的 NorFlash 中，观察串口输出；
- θ 打开实验例程目录\04-uCOS\3.3_keyboard_test 子 目录下的 ucos2.Uv2 例程，编译链接工程；
- 下载调试，观察结果。

2. 设计实现一个多任务应用程序

- 拷贝示例实验源码工程 3.3_keyboard_test；
- 添加 LCD 驱动程序（参考 11_LCD_Test）；
- 设计多任务。至少 LCD 显示用一个任务，键盘解析用一个。

五、实验结果

根据实验要求和步骤，首先将两个源码：3.1_LED_test 下的 ucos2.uv2 和 3.3_keyboard_test 下的 ucos2.Uv2 进行跑通，编译成功结果如下图所示：

D:\2021-2022学年\嵌入式\嵌入式实验\程序源码\04-UCOS\3.3_keyboard_test\ucos2.uvproj - µVision

File Edit View Project Flash Debug Peripherals Tools SVCS Window Help

Project: ucos2
- ucos
  - ucos
  - bsp
    - InitPort.c
    - InitStack.s
    - IRQ_ISR.s
    - Startup.s
    - Timer4.c
    - vector.s
    - MMU.s
    - Mem_CFG.s
    - 2410lib.c
    - iic_keybd.c
    - keyboard_test.c
    - lcd.c
    - ascii6x8.c
    - ascii8x16.c
    - bmp_data.c
    - color_lcd_test.c
    - hzk12.c
    - hzk16.c
    - hzk24.c
    - hzk24s.c
  - app
    - Main.c
      - 2410lib.h
      - ctype.h
      - Function.h
      - Includes.h
      - os_cfg.h
      - os_cpu.h
      - s3c2410_cfg.h
      - s3c2410sfr.h
      - stdio.h
      - stdlib.h
      - string.h
      - ucos_ii.h
      - Variable.h
  - s3c2410port
  - readme

Main.c

```c
    if(ch == 'B')
      number2 = 0;                                  // B :
    if(ch == 'F')
    {
        OSSemPost(num1_ready);
        OSSemPost(oper_ready);
        if(oper == '+') ans = number1 + number2;
        if(oper == '*') ans = number1 * number2;
        if(oper == '-') ans = number1 - number2;
        break;
    }                                               // F :
  }
}

/*****************************************************
* 函数: void led(void *Id).
* 描述: led函数.
*****************************************************
void led(void *Id)
{
    /* print task's id */
  OSSemPend(UART_sem, 0, &err);
  uart_sendstring("  led Called.\r\n");
  OSSemPost(UART_sem);

    while(1)
    {
    uart_sendstring("  led Called.\r\n");
    led_on();
    OSTimeDly(50);
    led_off();
    OSTimeDly(50);
  }
}

/*****************************************************
* 函数: void Display(void *Id).
* 描述: 展示结果函数.
*****************************************************
void Display(void *Id)
{
    /* print task's id */
  OSSemPend(UART_sem, 0, &err);
  uart_sendstring("  Display Called.\r\n");
  OSSemPost(UART_sem);

    while(1)
    {
    color_lcd_test(number1,number2,ans,oper);     /
    uart_sendstring("  Display Called.\r\n");
    OSTimeDly(30);
  }
```

Build Output

```
Program Size: Code=44412 RO-data=493308 RW-data=140 ZI-data=27656
Finished: 0 information, 1 warning and 0 error messages.
".\output\ucos2.axf" - 0 Error(s), 14 Warning(s).
Build Time Elapsed:  00:00:05
```

D:\2021-2022学年\嵌入式\嵌入式实验\程序源码\04-UCOS\3.1_LED_test\ucos2.Uv2 - µVision

File Edit View Project Flash Debug Peripherals Tools SVCS Window Help

Project: ucos2
- ucos
  - ucos
  - bsp
    - InitPort.c
    - InitStack.s
    - IRQ_ISR.s
    - Startup.s
    - Timer4.c
    - vector.s
    - MMU.s
    - Mem_CFG.s
    - 2410lib.c
  - app
    - Main.c
  - s3c2410port
  - readme

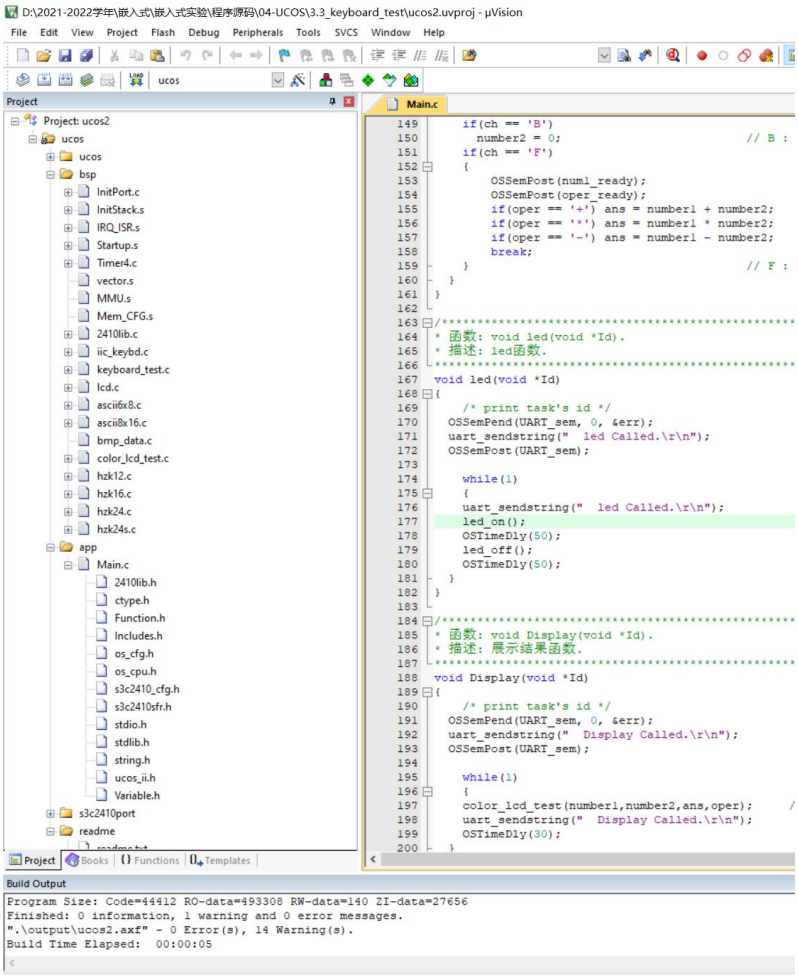
Main.c

```c
/*****************************************************
* 文件: Main.C
* 描述:application route
* 编写:
*****************************************************

#include  "Includes.h"
#include  "2410lib.h"

OS_EVENT *UART_sem;
OS_EVENT *InterruptSem;

unsigned char err;

/* allocate memory for tasks' stacks */
#define STACKSIZE 128

/* Global Variable */
OS_STK Stack1[STACKSIZE];
OS_STK Stack2[STACKSIZE];
OS_STK Stack3[STACKSIZE];
OS_STK StackMain[STACKSIZE];

const char Id1 = '1';
const char Id2 = '2';
const char Id3 = '3';


unsigned char err;

/*****************************************************

void led_init()
{
  rGPBCON  |= ((0x1<<14)| (0x1<<16)|(0x1<<18)|(0x1<<
  rGPBCON  &= ~((0x1<<15)| (0x1<<17)|(0x1<<19)|(0x1<
}


 void led_off()
 {
   rGPBDAT  |= ((0x1<<7)| (0x1<<8)|(0x1<<9)|(0x1<<10
  // delay(500);
}

void led_on()
{
  rGPBDAT  &= ~((0x1<<7)| (0x1<<8)|(0x1<<9)|(0x1<<10
//  delay(500);
```

Build Output

```
compiling os_dbg.c...
linking...
.\output\ucos2.sct(8): warning: L6314W: No section matches pattern *(InRoot$$Sections).
Program Size: Code=37716 RO-data=776 RW-data=108 ZI-data=23560
Finished: 0 information, 1 warning and 0 error messages.
".\output\ucos2.axf" - 0 Error(s), 6 Warning(s).
Build Time Elapsed:  00:00:03
```

我设计了 5 个任务，具体解释上文已解释，编译成功如图所示：



六、程序说明

程序核心代码如下：

```
/*****************************************************************************
**************
* 函数: void Get_Num1(void *Id).
* 描述: 任务 1 函数.
*****************************************************************************
*************/
int number1 = 0;
int number2 = 0;
char oper = '?';
int ans = 0;                          // 初始化两个运算数和操作符和结果
void Get_Num1(void *Id)
{
    /* print task's id */
    OSSemPend(UART_sem, 0, &err);
```

```
    uart_sendstring("  Get Num1 Called.\r\n");
    OSSemPost(UART_sem);
    OSSemPend(num1_ready,0,&err);
    while(1)
    {
        char ch;
    ch = keyboard_test();
        OSTimeDly(1);
        if(ch >= '0' && ch <= '9')
            number1 += number1*10+ ch - '0';   // 记录操作数 1，十进制
        if(ch == 'A')
            number1 /= 10;          // A : 删除上一位
        if(ch == 'B')
            number1 = 0;                // B : 重新输入
        if(ch == 'F')
        {
                OSSemPost(num1_ready);
              break;
        }                   // F : 结束输入，释放信号量，这样，操作符任务才可以进行
    }

}
/*******************************************************************************
**************
* 函数: void Get_Oper(void *Id).
* 描述: 获取操作符
*******************************************************************************
*************/
void Get_Oper(void *Id)
{
    /* print task's id */
    OSSemPend(UART_sem, 0, &err);
    uart_sendstring("  Get Oper Called.\r\n");
    OSSemPost(UART_sem);

    OSSemPend(oper_ready,0,&err);
    while(1)
    {
```

```
        char ch;
    ch = keyboard_test();
        OSTimeDly(1);
        if(ch == '+' || ch == '-' || ch == '*')
            oper = ch;                                  // 记录操作数 1，十进制
        if(ch == 'B')
            oper = '?';                                 // B : 重新输入
        if(ch == 'F')
        {
                OSSemPost(oper_ready);
              break;
        }                   // F : 结束输入，释放信号量，这样操作数 2 获取任务才可以进行
    }
}
/*******************************************************************************
**************
* 函数: void Get_Num2(void *Id).
* 描述: 获取操作数 2
*******************************************************************************
*************/
void Get_Num2(void *Id)
{
    /* print task's id */
    OSSemPend(UART_sem, 0, &err);
    uart_sendstring("  Get Num2 Called.\r\n");
    OSSemPost(UART_sem);
    OSSemPend(num1_ready,0,&err);
    OSSemPend(oper_ready,0,&err);
    while(1)
    {
        char ch;
    ch = keyboard_test();
        OSTimeDly(1);
        if(ch >= '0' && ch <= '9')
            number2 += number2*10+ ch - '0';            // 记录操作数 1，十进制
        if(ch == 'A')
            number2 /= 10;                      // A : 删除上一位
        if(ch == 'B')
```

```
            number2 = 0;                    // B : 重新输入
        if(ch == 'F')
        {
                OSSemPost(num1_ready);
                OSSemPost(oper_ready);
              if(oper == '+') ans = number1 + number2;
                if(oper == '*') ans = number1 * number2;
                if(oper == '-') ans = number1 - number2;
              break;
        }                   // F : 结束输入，释放信号量。
    }
}

/*****************************************************************************
************
* 函数: void led(void *Id).
* 描述: led 函数.
*****************************************************************************
************/
void led(void *Id)
{
    /* print task's id */
    OSSemPend(UART_sem, 0, &err);
    uart_sendstring("  led Called.\r\n");
    OSSemPost(UART_sem);

    while(1)
    {
        uart_sendstring("  led Called.\r\n");
        led_on();
        OSTimeDly(50);
        led_off();
        OSTimeDly(50);
    }
}

/*****************************************************************************
************
```

```
* 函数: void Display(void *Id).
* 描述: 展示结果函数.
*********************************************************************************
************/
void Display(void *Id)
{
    /* print task's id */
    OSSemPend(UART_sem, 0, &err);
    uart_sendstring("  Display Called.\r\n");
    OSSemPost(UART_sem);

    while(1)
    {
        color_lcd_test(number1,number2,ans,oper);          // lcd 展示
        uart_sendstring("  Display Called.\r\n");
        OSTimeDly(30);
    }
}
```

七、心得体会

通过这次实验，我理解了任务管理的基本原理，感受到了信号量在任务同步和异步执行的重大作用，通过程序可以理解了任务的各个基本状态及其变迁过程，同时实践操作掌握了嵌入式系统中 LCD 与键盘控制的一般方法，自己独立完成的计算器功能锻炼了从任务设计到代码编写与调试的实践过程，从中收获了很多。